Cloud handoff — #485, journalling one fetch is quadratic in the refs it moved

Written: 2026-08-25 · **By:** max (CLI session on Tom's box) · **For:** a cloud Claude Code session on `tom2025b/git-vista`

The measurements are already done, by the session that filed this. **A 500-ref fetch spends 27.6 seconds journalling, on the user's latency, and leaves a 34.7 MB journal.** You do not need to re-derive that. You need to fix it and prove the numbers moved.

```
task_id: gv-485-journal-quadratic
issue: 485
milestone: —          # a #329 successor
repo: tom2025b/git-vista
base: main
branch: fix/485-journal-capture-refs
sign_commits_as:
  name: Claude_Max
  email: 262510778+tom2025b@users.noreply.github.com
sign_artifacts_as: max
adr_number: 0080          # ASSIGNED. Do not pick "the next free" c
allowed_paths:
  - crates/git-vista-server/src/journal.rs
  - crates/git-vista-server/src/planner/fetch.rs
  - crates/git-vista-server/src/handlers/mod.rs
  - crates/git-vista-core/src/activity.rs
  - docs/adr/
forbidden_paths:
  - design-docs/
  - ci/browser/**
  - crates/git-vista/src/**
  - handoff.md
merge_order: independent — touches no file the stash follow-ups tou
```

The cause, in one paragraph

`fetch::journal_updates` (`planner/fetch.rs`) loops over the updated refs calling `journal_app_event` once per ref. That reaches `journal::append` (`journal.rs:124`), and because the event arrives with `refs: None` (`handlers/mod.rs:78`), `append` calls `capture_refs` (`journal.rs:96`) on **every iteration**. `capture_refs` does a full ref read of the repository and embeds up to `REFS_PER_EVENT_CAP` (500

— `activity.rs:260`) branches, 500 tags **and** 500 remote-tracking refs into that single line. So a fetch of N refs performs N full ref reads and writes N lines whose size itself grows with N.

The filed measurements, against real `git init` repos and the real `journal::append`:

refs moved	journal bytes	bytes/line	journalling time
1	538 B	537	2.4 ms
94	526.8 KiB	5,736	1,086.7 ms
500	14.1 MiB	28,872	**27,606.2 ms**

`bytes/line` going 537 → 28,872 is the quadratic, stated in one column.

And it is on the hot path, not in the background. `journal_updates(...).await` at `fetch.rs:333` completes before the executor returns `FetchStep::Completed`.

What to be careful about

The 24 August work is not the bug and must not be undone. #464/#467/#468 made the journal *read* bounded, linear and single-copy, and they did exactly what they claimed. This issue is about what each line *contains*. Do not "optimise" the reader again.

Capture the refs ONCE per fetch, not once per ref is the obvious shape, and it is probably right — but the decision that needs writing down is what a journal line means afterwards. Today every line carries a full ref snapshot, and something downstream may be relying on that being per-line rather than

per-batch. `assemble_feed` is where to look. If the answer is "one snapshot for the batch, and the lines reference it", say so in ADR 0080 and say what reads it.

`capture_refs` embedding 500 remote-tracking refs is its own question. #487 (latent, low) is about push journalling per remote-tracking ref. If your fix makes #487 moot, say so on that issue rather than silently closing it.

What you cannot run

The browser leg does not run in a cloud container — the kernel reports `landlock_abi=-1` and INV-13 gives the server no degraded tier. Installing `bwrap` changes nothing. This task should not need it; if you find yourself wanting it, the change has reached the frontend, which is outside `allowed_paths`.

Two tests in `git-vista-server` flake under parallel execution because they race on the process-global current repository (#438): `recovery_center::tests::a_stale_claimed_undo_is_refused_and_the_branch_is_left_alone` and `state::tests::selection_flow_carries_mode_and_gates_writes`. Re-run before believing either is your doing.

Acceptance

1. `bytes/line` is flat in the number of refs moved. **Reproduce the filed table's method and put your own row for 500 refs in the PR body.** A claim that it is fixed, without the number beside the old one, is not this issue's deliverable.
2. Journalling time for a 500-ref fetch is a small multiple of the 1-ref case, not 11,000×.

3. A test that fails if the per-ref full ref read comes back — and proved able to go red **two different ways** (remove the batching, and weaken it to every-other-ref). One `caught` verdict is not proof.
4. **ADR 0080** records what a journal line carries after this change, and what reads it. `docs/adr/README.md` index updated.
5. `cargo fmt --all`, `cargo clippy --all-targets -- -D warnings`, `cargo test --workspace` green.
6. PR body says `Closes #485`. **Never delete the branch.**

Signed: max · 2026-08-25T10:25:00-04:00